SESSION 7

Date Analysis, Subqueries, CTEs & Views

Build reusable SQL layers for time-based analytics and Power BI reporting

Session 7 — Overview

What you will learn and how the session is structured

INSTANT

01 **Date Functions**

GETDATE, DATEADD, DATEDIFF, YEAR, MONTH, DAY, FORMAT — analyze enrollment timelines and payment trends

02 **Subqueries**

Nested SELECT statements in WHERE, FROM, and SELECT clauses — filter, compare, and enrich your queries

03 **CTEs**

Common Table Expressions (WITH) — break complex SQL into readable, reusable named steps

04 **Views**

CREATE VIEW — store reusable query logic for dashboards, Power BI, and team collaboration

🎯 **By the end: analyze time-based data, write layered SQL, and build reusable Views for Power BI dashboards**

Quick Recap: The Academy Dataset

Our business case — tables we'll use throughout this session

INSTANT

Students

PK StudentID

StudentID, StudentName, Email, City, EnrollDate, Status, NationalID

Courses

PK CourseID

CourseID, CourseName, Category, Price, DurationHrs, StartDate

Enrollments

PK EnrollmentID

EnrollmentID, StudentID, CourseID, EnrollDate, EnrollStatus

Payments

PK PaymentID

PaymentID, EnrollmentID, Amount, PaymentDate, Method, Status

Leads

PK LeadID

LeadID, LeadName, Source, AgentID, CampaignID, Status, Email, Phone, CreatedDate

Campaigns

PK CampaignID

CampaignID, AgentID, CampaignName, Channel, Budget, StartDate, EndDate, IsActive

Attendance

PK AttendanceID

AttendanceID, EnrollmentID, AttendStatus, Notes

SalesAgents

PK AgentID

AgentID, AgentName, Email, Phone, Region, MonthlyTarget, HireDate, Commission, IsActive

SESSION 7

Part 1 — Date Functions

Analyze time, trends, and timelines using SQL Server date functions

Why Date Analysis Matters

Time is one of the most powerful dimensions in business data



Monthly Revenue

Track payment trends



Enrollment Timing

Peak seasons & cohorts



Agent Performance

Leads over time periods

Business Questions We Can Answer with Date Functions:

- How many students enrolled this month vs. last month?
- Which courses had the most enrollments in Q1 2024?
- How many days have passed since a lead was created?
- What is the revenue trend over the past 6 months?
- Which agents have been active for more than 2 years?

SQL Server Date Functions — Reference

Core functions for extracting and manipulating date values

Function	Purpose	Example	Result
GETDATE()	Current date & time	`GETDATE()`	2024-03-15 10:30:00
YEAR(date)	Extract year number	`YEAR(EnrollDate)`	2024
MONTH(date)	Extract month number	`MONTH(PaymentDate)`	3
DAY(date)	Extract day number	`DAY(HireDate)`	15
DATEADD(part,n,date)	Add/subtract time	`DATEADD(MONTH,-1,GETDATE())`	Last month
DATEDIFF(part,d1,d2)	Difference between dates	`DATEDIFF(DAY,HireDate,GETDATE())`	Days employed
FORMAT(date,'format')	Format as string	`FORMAT(EnrollDate,'MMM yyyy')`	Mar 2024
EOMONTH(date)	Last day of month	`EOMONTH(GETDATE())`	2024-03-31
CAST(date AS DATE)	Strip time portion	`CAST(PaymentDate AS DATE)`	2024-03-15

Date Extraction — YEAR, MONTH, DAY

Extract specific parts of a date column to group and filter data

📌 Academy Scenario: Count monthly student enrollments for 2024 — group by year and month

```sql
-- Monthly enrollment count per year

SELECT
    YEAR(EnrollDate)    AS EnrollYear,
    MONTH(EnrollDate)   AS EnrollMonth,
    COUNT(*)            AS TotalEnrollments
FROM  Enrollments
WHERE  YEAR(EnrollDate) = 2024
GROUP BY  YEAR(EnrollDate), MONTH(EnrollDate)
ORDER BY  EnrollYear, EnrollMonth;
```

💡 Result: A row per month — e.g., 2024 | 1 | 47 enrollments → 2024 | 2 | 63 enrollments — perfect for trend charts in Power BI

DATEDIFF & DATEADD — Time Calculations

Calculate durations and shift dates forward or backward

INSTANT

DATEDIFF — How long ago?

```
-- Days since each lead was created

SELECT
  LeadName,
  CreatedDate,
  DATEDIFF(DAY,
    CreatedDate, GETDATE())
    AS DaysSinceCreated
FROM  Leads;
```

DATEADD — Shift a date

```
-- Payments in the last 30 days

SELECT
  PaymentID, Amount,
  PaymentDate
FROM  Payments
WHERE  PaymentDate >=
  DATEADD(DAY,-30,
    GETDATE());
```

Date Part Reference:

YEAR	— Annual shifts or diffs	QUARTER	— Business quarters	MONTH	— Monthly reporting
WEEK	— Weekly cohorts	DAY	— Daily lead follow-up	HOUR	— Intraday analysis

FORMAT — Display Dates for Reporting

Convert date values into readable labels for dashboards and reports

```sql
-- Format payment dates for reporting labels

SELECT
    PaymentID,
    Amount,
    FORMAT(PaymentDate, 'MMM yyyy')    AS PaymentMonth,    -- e.g. 'Mar 2024'
    FORMAT(PaymentDate, 'dd/MM/yyyy') AS FormattedDate,   -- e.g. '15/03/2024'
    FORMAT(PaymentDate, 'dddd')        AS DayName          -- e.g. 'Friday'
FROM  Payments
ORDER BY  PaymentDate;
```

Common Format Codes:

`'yyyy'`	→ 2024	`'MMM'`	→ Mar	`'MMMM'`	→ March
`'dd/MM/yyyy'`	→ 15/03/2024	`'MMM yyyy'`	→ Mar 2024	`'dddd'`	→ Friday

Practice Exercise — Date Functions

Apply date functions to real Instant Academy scenarios

⚡ **Exercise 1 — Complete all 4 tasks using the Instant Academy database**

1 List all students who enrolled in 2024. Show StudentName, EnrollDate, and the month name using FORMAT.

2 Calculate how many days each lead has been in the system (from CreatedDate to today). Show LeadName and DaysSinceCreated.

3 Show all payments made in the last 60 days. Display Amount and a formatted date label (e.g. 'Mar 2024').

4 Count enrollments grouped by YEAR and MONTH for the Enrollments table. Order by most recent first.

SESSION 7

Part 2 — Subqueries

Nest SELECT statements to filter, compare, and enrich your results

What Is a Subquery?

A SELECT statement nested inside another SELECT statement

A subquery is a query written inside parentheses ( ) that the outer query uses as a data source, a filter value, or a calculated column. SQL runs the inner query first, then uses its result in the outer query.



In WHERE

Filter rows based on the result of an inner query

WHERE StudentID IN (SELECT ...)



In FROM

Treat the subquery result as a temporary table

FROM (SELECT ...) AS SubName



In SELECT

Calculate a value for each row using a correlated query

SELECT (SELECT COUNT(*) ...)

Subquery in WHERE Clause

Use a subquery to filter rows dynamically

📌 Scenario: Find all students who have at least one enrollment in a 'Data Analysis' course

```
-- Subquery in WHERE: Find students enrolled in Data Analysis courses

SELECT
    StudentID,
    StudentName,
    Email
FROM  Students
WHERE  StudentID IN (
    SELECT  e.StudentID
    FROM    Enrollments e
    JOIN    Courses c ON e.CourseID = c.CourseID
    WHERE   c.Category = 'Data Analysis'
);
```

How it works: SQL runs the inner query (green) first → gets a list of StudentIDs → the outer query then selects only those students. The inner query runs **once** and its result is used as a filter list.

Subquery in FROM Clause

Use a subquery as a virtual table — also called a derived table

📌 Scenario: Find agents whose total lead count is above the academy average

```
-- Subquery in FROM: agents above average lead count

SELECT  AgentName, LeadCount
FROM  (
    SELECT
        a.AgentName,
        COUNT(l.LeadID) AS LeadCount
    FROM    SalesAgents a
    LEFT JOIN Leads l ON a.AgentID = l.AgentID
    GROUP BY a.AgentName
) AS AgentLeads
WHERE  LeadCount > (SELECT AVG(LeadCount) FROM (...same subquery...) AS x);
```

⚠️ Note: Every subquery in FROM must have an alias (AS AgentLeads). Without the alias, SQL Server will return an error.

Scalar Subquery in SELECT

Return a single calculated value for each row

```sql
-- For each course, show its price vs the average price across all courses

SELECT
    CourseName,
    Price,
    (SELECT AVG(Price) FROM Courses)          AS AvgPrice,
    Price - (SELECT AVG(Price) FROM Courses)  AS DiffFromAvg
FROM  Courses
ORDER BY  DiffFromAvg DESC;
```

When to use scalar subqueries:

- Compare each row's value against an overall aggregate (AVG, MAX, MIN)
- Show a benchmark or target alongside actual data without a JOIN

⚠️ **Must return exactly ONE value — if it returns multiple rows, SQL Server will throw an error**

Practice Exercise — Subqueries

Write nested queries to answer real academy business questions

⚡ **Exercise 2 — Use subqueries (IN, FROM, SELECT clause) to solve these tasks**

1 WHERE IN

Find all payments made by students who live in Cairo. Use a subquery on the Students table to get StudentIDs first.

2 FROM

Using a derived table, show each course's name and enrollment count. Then filter to show only courses with more than 10 enrollments.

3 SELECT

For each campaign, show the CampaignName, Budget, and the total number of leads linked to it. Include the overall average budget as a comparison column.

4 Challenge

List all agents whose number of converted leads (Status = 'Converted') is above the agent average. Use a subquery in WHERE or FROM.

SESSION 7

Part 3 — CTEs

Common Table Expressions — readable, reusable named query blocks

What Is a CTE?

Common Table Expressions make complex SQL readable and maintainable

A CTE (Common Table Expression) is a named temporary result set defined with the WITH keyword. It exists only for the duration of the query. Think of it as giving a name to a subquery so you can reference it cleanly — like creating a virtual table on the fly.

```sql
-- CTE Syntax Template
WITH  CTEName AS (
    -- Your inner query goes here
    SELECT  column1, column2 FROM SomeTable WHERE condition
)
SELECT  *  FROM  CTEName;
```

❌ Subquery (hard to read)

```sql
SELECT * FROM
  (SELECT AgentID, COUNT(*) AS Total
   FROM Leads GROUP BY AgentID) x
WHERE x.Total > 10
```

✅ CTE (clean & readable)

```sql
WITH AgentTotals AS (
  SELECT AgentID, COUNT(*) AS Total
  FROM Leads GROUP BY AgentID
)
SELECT * FROM AgentTotals
WHERE Total > 10
```

CTE Example — Monthly Revenue Summary

Use a CTE to build a clean monthly revenue report from Payments

📌 Scenario: Calculate monthly revenue, then find which months exceeded the 6-month average

```
-- Step 1: CTE calculates monthly revenue
WITH  MonthlyRevenue AS (
    SELECT
        YEAR(PaymentDate)              AS PayYear,
        MONTH(PaymentDate)             AS PayMonth,
        FORMAT(PaymentDate, 'MMM yyyy') AS MonthLabel,
        SUM(Amount)                    AS TotalRevenue
    FROM   Payments
    WHERE  Status = 'Completed'
    GROUP BY YEAR(PaymentDate), MONTH(PaymentDate), FORMAT(PaymentDate,'MMM yyyy')
)
-- Step 2: Select from CTE and compare to average
SELECT  MonthLabel, TotalRevenue,
        AVG(TotalRevenue) OVER () AS AvgRevenue
FROM    MonthlyRevenue
ORDER BY PayYear, PayMonth;
```

Multiple CTEs — Chain Your Logic

Define several named steps and join them together

INSTANT

📌 Scenario: For each agent, show total leads AND total conversions in one clean report

```
WITH
-- CTE 1: Count all leads per agent
AgentLeads AS (
    SELECT AgentID, COUNT(*) AS TotalLeads
    FROM   Leads  GROUP BY AgentID
),
-- CTE 2: Count only converted leads per agent
AgentConversions AS (
    SELECT AgentID, COUNT(*) AS Conversions
    FROM   Leads WHERE Status = 'Converted'
    GROUP BY AgentID
)
-- Final: Join both CTEs to SalesAgents
SELECT  a.AgentName, l.TotalLeads, c.Conversions,
        ROUND(c.Conversions * 100.0 / l.TotalLeads, 1) AS ConvRate
FROM    SalesAgents a
JOIN    AgentLeads l       ON a.AgentID = l.AgentID
JOIN    AgentConversions c ON a.AgentID = c.AgentID
ORDER BY ConvRate DESC;
```

Practice Exercise — CTEs

Structure complex queries using Common Table Expressions

⚡ **Exercise 3 — Use WITH … AS to organize your queries into named steps**

1 Write a CTE named CourseEnrollments that counts enrollments per course. Then SELECT from it to show only courses with more than 5 enrollments.

2 Create a CTE named RecentPayments that filters Payments from the last 90 days with Status = 'Completed'. Then calculate total revenue and average payment from that CTE.

3 Use two CTEs: one for total leads per campaign (CampaignLeads), and one for total budget per campaign from the Campaigns table. Join them to show ROI = Leads / Budget.

4 Challenge: Build a CTE that shows each student's latest enrollment date. Then join it to the Students table and show students who enrolled within the last 6 months.

SESSION 7

Part 4 — Views

Store reusable SQL logic as named database objects for dashboards and teams

What Is a View?

A saved SELECT query that behaves like a virtual table

A View is a stored SELECT query saved in the database with a name. Once created, you can query it like a regular table — SELECT * FROM ViewName. The data is always live (no snapshot). Views are ideal for Power BI, shared team reporting, and hiding complex joins.



Reusability

Write once, query many times — no copy-paste SQL



Security

Grant access to the view without exposing base tables



Power BI Ready

Connect Power BI directly to views like tables



Simplicity

Hide complex joins from analysts and report users

CREATE VIEW — Syntax & Lifecycle

How to create, use, modify, and drop views

```sql
-- Create a view
CREATE VIEW vw_StudentEnrollments AS
    SELECT s.StudentName, c.CourseName, e.EnrollDate, e.EnrollStatus
    FROM   Enrollments e
    JOIN   Students s ON e.StudentID = s.StudentID
    JOIN   Courses  c ON e.CourseID  = c.CourseID;

-- Use the view exactly like a table
SELECT * FROM vw_StudentEnrollments WHERE EnrollStatus = 'Active';
```

Create	Modify	Delete	Query
`CREATE VIEW name AS`	`ALTER VIEW name AS`	`DROP VIEW name`	`SELECT * FROM name`
Define the view for the first time	Update the view's query logic	Remove the view from the database	Use it like any regular table

Academy Views — Real Examples

Useful views for the Instant Academy reporting layer

INSTANT

vw_MonthlyRevenue

Monthly payment summary for revenue dashboards

```
CREATE VIEW vw_MonthlyRevenue AS
SELECT FORMAT(PaymentDate,'MMM yyyy') AS Month,
       SUM(Amount) AS Revenue
FROM   Payments WHERE Status='Completed'
GROUP BY FORMAT(PaymentDate,'MMM yyyy'),
         YEAR(PaymentDate), MONTH(PaymentDate);
```

vw_AgentPerformance

Leads and conversion rates per sales agent

```
CREATE VIEW vw_AgentPerformance AS
SELECT a.AgentName, a.Region,
       COUNT(l.LeadID)    AS TotalLeads,
       SUM(CASE WHEN l.Status='Converted'
           THEN 1 ELSE 0 END) AS Conversions
FROM SalesAgents a LEFT JOIN Leads l
ON a.AgentID=l.AgentID GROUP BY a.AgentName,a.Region;
```

Views → Power BI Integration

Connect Power BI directly to SQL Server views as data sources



Why connect Power BI to Views (not raw tables)?

✅ Views pre-aggregate and pre-join data — faster report loading

✅ Business logic stays in SQL — not scattered in Power Query

✅ Security: analysts see only the columns the view exposes

To connect in Power BI Desktop: Home → Get Data → SQL Server → Server name → Database → Select your vw_ views → Load

Practice Exercise — Views

Create and query views for the Instant Academy reporting layer

⚡ **Exercise 4 — Create SQL Server views and query them like tables**

1 Create vw_ActiveStudents: a view that shows all students where Status = 'Active'. Include StudentName, City, and EnrollDate. Then SELECT all from it.

2 Create vw_CourseRevenue: a view showing CourseName and total revenue (from Payments via Enrollments). Only include completed payments. Query it to find top 5 courses by revenue.

3 Create vw_LeadSummary: per agent — total leads, leads this month, and conversion rate. Use DATEDIFF or MONTH() to filter current-month leads.

4 Bonus: ALTER your vw_CourseRevenue to also include the number of enrollments per course. Then re-query it to show courses where Revenue > 50,000.

Bringing It All Together

Combine date functions + CTEs + views in one end-to-end query

📌 Scenario: Build a monthly agent performance view using CTEs and date functions for Power BI

```
CREATE VIEW vw_AgentMonthlyKPIs AS
WITH
MonthlyLeads AS (                   -- CTE 1: leads per agent per month
    SELECT  AgentID,
            YEAR(CreatedDate)               AS LeadYear,
            MONTH(CreatedDate)              AS LeadMonth,
            FORMAT(CreatedDate,'MMM yyyy') AS MonthLabel,
            COUNT(*)                        AS TotalLeads,
            SUM(CASE WHEN Status='Converted' THEN 1 ELSE 0 END) AS Converted
    FROM    Leads
    GROUP BY AgentID, YEAR(CreatedDate), MONTH(CreatedDate), FORMAT(CreatedDate,'MMM yyyy')
)
SELECT  a.AgentName, a.Region, m.MonthLabel,
        m.TotalLeads, m.Converted,
        ROUND(m.Converted*100.0/NULLIF(m.TotalLeads,0),1) AS ConvRate
FROM    MonthlyLeads m
JOIN    SalesAgents  a ON m.AgentID = a.AgentID;
```

Subquery vs CTE vs View — When to Use Each

Choose the right technique for the right situation

[i]NSTANT

	Subquery	CTE (WITH)	View
Definition	Inline nested SELECT inside ( )	Named block with WITH ... AS	Saved query in the database
Readability	Low — nested & hard to follow	High — named steps	Very high — query it by name
Reusability in query	Must repeat if needed again	Reference by name once defined	Reference anytime, forever
Persists in DB	No — runs once, then gone	No — per-query scope only	Yes — stored permanently
Power BI use	No — not accessible directly	No — query-level only	Yes — connect as a table
Best for	Quick one-off filters	Multi-step readable logic	Shared reporting & dashboards
Example	WHERE x IN (SELECT...)	WITH cte AS (...) SELECT...	SELECT * FROM vw_Revenue

Mini Project — Academy Analytics Layer

Build a complete SQL reporting layer for Power BI

INSTANT

🏆 Project Goal: Create 3 production-ready views that could be connected directly to Power BI for executive reporting at Instant Academy

1 `vw_MonthlyEnrollmentRevenue`

Monthly enrollments count + revenue from payments. Use YEAR(), MONTH(), FORMAT(), and JOIN Enrollments → Payments. Group by month.

2 `vw_AgentLeaderboard`

Per agent: TotalLeads, ConvertedLeads, ConversionRate%, MonthlyAvgLeads. Use DATEDIFF and CTEs before wrapping in the view.

3 `vw_CourseDemand`

Per course: enrollment count, total revenue, average payment, days since last enrollment (DATEDIFF). Join Courses, Enrollments, Payments.

💡 **Deliverable: A .sql script file with all 3 CREATE VIEW statements + 3 SELECT queries showing the view output**

Homework Assignment

Consolidate this session's skills with independent practice

[i]NSTANT

📝 **Submit a .sql file with all queries before next session — use the Instant Academy database**

Dates **Q1**

Show all students who enrolled in the last 6 months. Display StudentName, EnrollDate, and how many days ago they enrolled (DATEDIFF).

Dates **Q2**

Generate a quarterly revenue summary (Q1–Q4) for 2024. Use DATEPART(QUARTER, ...) to group payments by quarter.

Subquery **Q3**

Find courses that have NEVER had an enrollment. Use NOT IN with a subquery on the Enrollments table.

Subquery **Q4**

List students whose total payment amount (from Payments) is above the academy's average. Use a subquery in WHERE or HAVING.

CTE **Q5**

Write a CTE called TopCourses that finds the 5 courses with the most enrollments. Then join it back to Courses to show the full course details and price.

View **Q6**

Create vw_CampaignROI: shows each campaign's name, total budget, total leads generated, and cost-per-lead (Budget ÷ Leads). Use a LEFT JOIN.

Session 7 — Key Takeaways

What you can now do after this session

📅 Date Functions

- ✓ Extract YEAR, MONTH, DAY from any date column
- ✓ Calculate durations with DATEDIFF
- ✓ Shift dates with DATEADD
- ✓ Format labels with FORMAT()

🔍 Subqueries

- ✓ Filter with IN (subquery) in WHERE
- ✓ Build derived tables in FROM clause
- ✓ Return scalar values in SELECT clause
- ✓ Understand inner-then-outer execution

🧱 CTEs

- ✓ Use WITH … AS to name query steps
- ✓ Chain multiple CTEs with commas
- ✓ Improve readability over nested subqueries
- ✓ Reference CTE names in JOIN

🏗️ Views

- ✓ Create reusable saved queries
- ✓ ALTER and DROP views
- ✓ Connect views to Power BI
- ✓ Build a reporting layer for teams

SESSION 7 COMPLETE

Well done! You can now write time-aware, layered SQL queries and build reusable views.

NEXT → Session 8

Window Functions, Stored Procedures & Final Project

ROW_NUMBER, RANK, LAG/LEAD • OVER (PARTITION BY) • SP basics • Capstone dashboard